

Problem ID	Captured Flags	Steps
one_byte	fsuCTF{0h_n0_r3v3r5i813_3ncr yp7i0n}	1. Base64 2. Get key 3. Decipher
My Public Key	fsuCTF{A_Big_N_Does_Not_Mat ter_If_Its_Factors_Are_Known}	1. Get p and q 2. Solve Script
The Cooler Cipher	fsuCTF{You_Know_What_ROT13 _Aint_Cool_Enough_Have_ROT4 7}	1. ROT47 2. Cyberchef 3. Increase amount to 52

One_byte

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/cypher]
$ cat ciphertext.txt
IQYwJgc1NlUvKitVDAF+E3QHcAxrQn46dBsmFyoDegx3Gzg=

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/cypher]
$
```

We are given the following ciphertext. Immediately I think base64 because of the '=' sign at the end. I try to decode it that way but it did not work. I realize that I need a key to decipher this. I know that the first letters are going to be fsuCTF{ so I use that to find out the key. Using pwn tools I generate the following script:

```
Session Actions Edit View Help
from pwn import xor
import base64

ciphertext = base64.b64decode("IQYwJgc1NlUvKitVDAF+E3QHcAxrQn46dBsmFyoDegx3Gzg=")
known_prefix = b"fsuCTF{"

print(xor(ciphertext[:7], known_prefix))
~
```

This prints out the key to the cipher which is:

```
Interval=never
[*] You have the latest version of Pwntools (4.15.0)
b'GuEeSsM'
```

Using this we can now decipher the text using the following python script:

```
from pwn import xor
import base64

encoded_str = "IQYwJgc1NlUvKitVDAF+E3QHcAxrQn46dBsmFyoDegx3Gzg="

ciphertext = base64.b64decode(encoded_str)

key = b"GuEeSsMe"

flag = xor(ciphertext, key)

print(f"Flag: {flag.decode()}")
```

And we get the flag: fsuCTF{0h_n0_r3v3r5i813_3ncryp7i0n}

```
(.venv)-(kali㉿kali)-[~/CTFs/CTF_Class/cypher]
$ python3 sol.py
Flag: fsuCTF{0h_n0_r3v3r5i813_3ncryp7i0n}
```

```
(.venv)-(kali㉿kali)-[~/CTFs/CTF_Class/cypher]
$
```

My Public Key

```
p: REDACTED
q: REDACTED
n: 6533578208109509862979881021880908309292149268002743700796929530530033849340694579388440987065638511182899257588401176563544134568780495589980276573209831
phi: REDACTED
e: 65537
d: REDACTED
m = REDACTED
c = 2231932102428642830114215205252591013842413609743127437635523939112990724249058644794998942631435583827186541004896371367693520755048537406818767080482346
m = REDACTED
```

We are given the public key n and we need to get its factors p and q. So I go to a factorization database and get p and q:

6533578208109509862979881021880908309292149268002743700796929530530033849340694579388440987065638511182899257588401176563544134568780495589980276573209831

Result:	
number	6533578208109509862979881021880908309292149268002743700796929530530033849340694579388440987065638511182899257588401176563544134568780495589980276573209831

[6533578208109509862979881021880908309292149268002743700796929530530033849340694579388440987065638511182899257588401176563544134568780495589980276573209831](#) = [57972899731021880908309292149268002743700796929530530033849340694579388440987065638511182899257588401176563544134568780495589980276573209831](#) · [11270055901021880908309292149268002743700796929530530033849340694579388440987065638511182899257588401176563544134568780495589980276573209831](#)

```

from Crypto.Util.number import long_to_bytes

n = 6533578208109509862979881021880908309292149268002743700796929530530033849340694579388440987065638511182899257588401176563544134568780495589980276
e = 65537
c = 2231932102428642830114215205252591013042413609743127437635523939112990724249058644794998942631435583827186541004896371367693520755048537406818767

p = 57972899739301697246742988213541747352626973514664747016615561626688795315337
q = 112700559011026778313623222438110524958206042791305955170078924032539414202863

phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)
m = pow(c, d, n)

flag = long_to_bytes(m)
print(flag.decode())

```

I put all the given information at the top, my factored p and q and then decipher by getting the private key d by taking the modular inverse of the public exponent e modulo Euler's totient of n. And when this is ran we get the flag: fsuCTF{A_Big_N_Does_Not_Matter_If_Its_Factors_Are_Known}

```

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/cypher]
$ python3 sol.py
fsuCTF{A_Big_N_Does_Not_Matter_If_Its_Factors_Are_Known}

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/cypher]
$

```

The Cooler Cipher

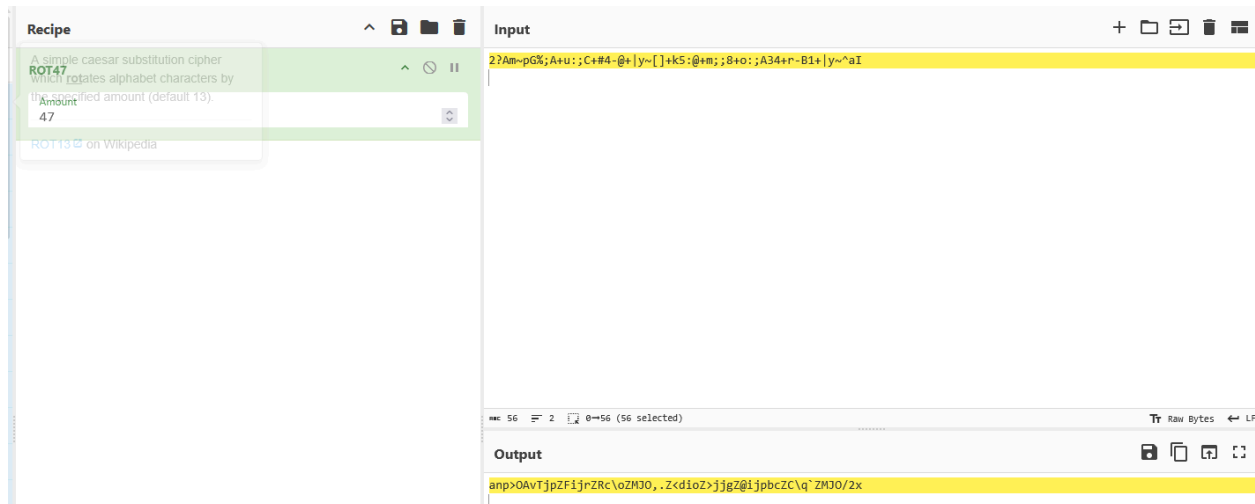
```

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/cypher]
$ cat ciphertext.txt
2?Am~pG%;A+u:;C+#4-@+|y~[]+k5:@+m;;8+o:;A34+r-B1+|y~^aI

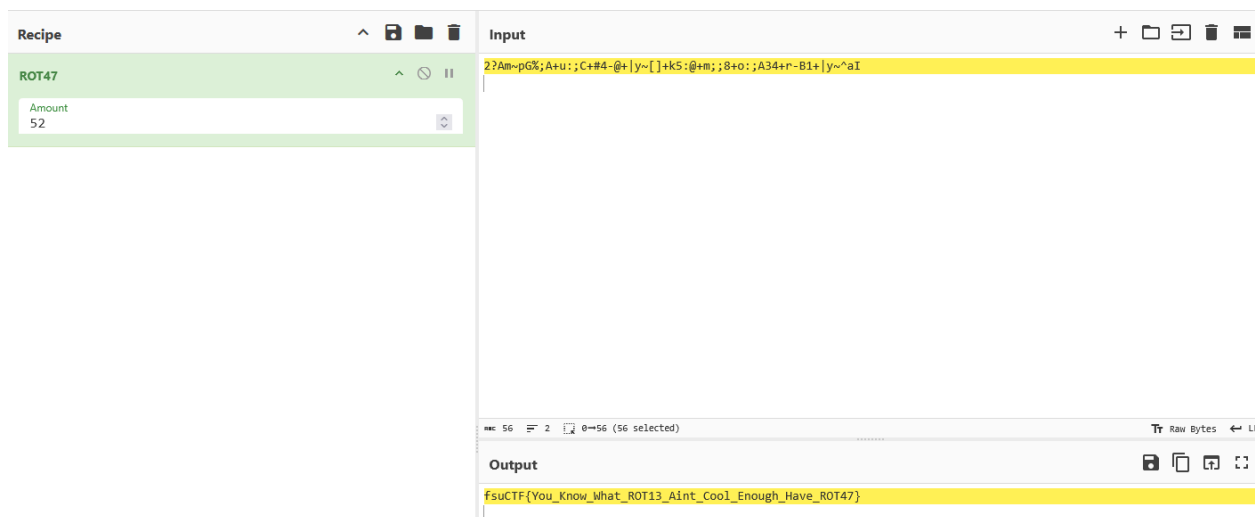
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/cypher]
$

```

I plugged this into Gemini and it said that it was ROT47 encoded which goes with the hint about ROT13. So I plug this into cyberchef:



And got nothing. However by experimenting with the amounts and increasing it to 52 I got the flag:



Flag: fsuCTF{You_Know_What_ROT13_Aint_Cool_Enough_Have_ROT47}